

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Nombre del Curso				
TÍTULO DE LA PRÁCTICA:	Práctica 01				
NÚMERO DE LA PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	29/09/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s):	- Hanco Mullisaca Sergio Danilo			NOTA (0 - 20):	Nota colocada por el docente
DOCENTE:	Nombre del Docente				

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE Enunciado: Construir una aplicación con tres pantallas (Home, Edificios y Mapa) navegables desde una barra inferior de navegación, comunicando la selección de un edificio mediante una función lambda. Desarrollo Paso 1. Creación del proyecto y definición de la sealed class Screen Se creó el proyecto en Android Studio y se definió la clase sellada Screen con sus tres objetos, cada uno con su propiedad route (String) y label (String): Además se agregaron las importaciones que se utilizan en los siguientes pasos: <pre> 1 import android.os.Bundle 2 import androidx.activity.ComponentActivity 3 import androidx.activity.compose.setContent 4 import androidx.compose.foundation.layout.Box 5 import androidx.compose.foundation.layout.Column 6 import androidx.compose.foundation.layout.Row 7 import androidx.compose.foundation.layout.fillMaxSize 8 import androidx.compose.foundation.layout.fillMaxWidth 9 import androidx.compose.foundation.layout.padding 10 import androidx.compose.foundation.lazy.LazyColumn 11 import androidx.compose.foundation.lazy.items 12 import androidx.compose.material.icons.Icons 13 import androidx.compose.material.icons.filled.Home 14 import androidx.compose.material.icons.automirrored.filled.List 15 import androidx.compose.material.icons.filled.Place 16 import androidx.compose.material3.Button 17 import androidx.compose.material3.Icon 18 import androidx.compose.material3.NavigationBar 19 import androidx.compose.material3.NavigationBarItem 20 import androidx.compose.material3.Scaffold 21 import androidx.compose.material3.Text 22 import androidx.compose.runtime.Composable 23 import androidx.compose.runtime.getValue 24 import androidx.compose.runtime.mutableStateOf 25 import androidx.compose.runtime.remember 26 import androidx.compose.ui.Alignment 27 import androidx.compose.ui.Modifier 28 import androidx.compose.ui.unit.dp 29 import androidx.navigation.compose.NavHost 30 import androidx.navigation.compose.composable 31 import androidx.navigation.compose.currentBackStackEntryAsState 32 import androidx.navigation.compose.rememberNavController 33 import com.example.campusnavigator_compose.ui.theme.CampusNavigator_ComposeTheme 34 </pre> Figura 1: Paso 1. Configuración de importaciones y definición de la sealed class Screen. Paso 2. Dependencias necesarias en build.gradle.kts Se verificó el archivo build.gradle.kts (Module :app) y se confirmaron las dependencias requeridas dentro del bloque dependencies { ... }:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

La librería `androidx.navigation:navigation-compose` aporta el `NavHost` y el `NavController`, mientras que `androidx.compose.material:material-icons-extended` provee los iconos `Icons.Default.Home`, `Icons.Default.List` e `Icons.Default.Place` de la barra de navegación. Una vez agregadas, se ejecutó la sincronización del proyecto con el botón **Sync Now** antes de continuar.

```
dependencies {
    implementation(platform("dependencyProvider = libs.androidx.compose.bom"))
    implementation(libs.androidx.activity.compose)
    implementation(libs.androidx.compose.material3)
    implementation(libs.androidx.compose.ui)
    implementation(libs.androidx.compose.ui.graphics)
    implementation(libs.androidx.compose.ui.tooling.preview)
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    testImplementation(libs.junit)
    androidTestImplementation(platform("dependencyProvider = libs.androidx.compose.bom"))
    androidTestImplementation(libs.androidx.compose.ui.test.junit4)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(libs.androidx.junit)
    debugImplementation(libs.androidx.compose.ui.test.manifest)
    debugImplementation(libs.androidx.compose.ui.tooling)
    implementation(libs.androidx.navigation.compose)
    implementation(libs.androidx.compose.material.icons.extended)}
}
```

Figura 2: Paso 2. Dependencias añadidas en `build.gradle.kts` y sincronización del proyecto.

Paso 3. Composable raíz con `NavController` y estado del edificio

En el Composable raíz se creó el `NavController` y la variable de estado que almacenará el último edificio consultado:

`rememberNavController()` crea el controlador de navegación persistente entre recomposiciones, mientras que `mutableStateOf("Ninguno")` inicializa el estado con el valor por defecto «Ninguno» y permite actualizarlo mediante la lambda.

```
1 Usage
60 @Composable
61 fun MainScreen() {
62
63     val navController = rememberNavController()
64     var edificioSeleccionado by remember { mutableStateOf(value = "Ninguno") }
65 }
```

Figura 3: Paso 3. Declaración de `NavController` y variable de estado `edificioSeleccionado`.

Paso 4. Scaffold con la `NavigationBar` inferior

Se envolvió la interfaz dentro de un `Scaffold(bottomBar = { ... })` para estructurar la pantalla. En el `bottomBar` se declaró una `NavigationBar` con un `NavigationBarItem` por cada `Screen`, marcando como seleccionado el que coincide con la ruta actual obtenida mediante `currentBackStackEntryAsState()`, y llamando a `navController.navigate(screen.route)` al pulsarlo:

De esta forma la barra inferior controla qué ítem se muestra resaltado según la ruta activa de la pila de navegación.

```
1 Usage
2 Scaffold
3 Fun MainScreen() {
4     val navController = rememberNavController()
5     var edificioSeleccionado by remember { mutableStateOf(value = "Ninguno") }
6     val items = listOf(Screen.Home, Screen.Edificios, Screen.Mapa)
7     Scaffold(
8         bottomBar = {
9             NavigationBar {
10                 val navBackStackEntry by navController.currentBackStackEntryAsState()
11                 val currentRoute = navBackStackEntry?.destination?.route
12                 items.forEach { screen ->
13                     val icon = when (screen) {
14                         Screen.Home ->
15                             Icons.Default.Home
16                         Screen.Edificios ->
17                             Icons.Default.List
18                         Screen.Mapa ->
19                             Icons.Default.Place
20                     }
21                     NavigationBarItem(
22                         selected = currentRoute == screen.route,
23                         onClick = {
24                             navController.navigate(screen.route)
25                         }
26                     )
27                 }
28             }
29         }
30     )
31 }
```

Figura 4: Paso 4. Scaffold con `NavigationBar` y `NavigationBarItems`.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

Paso 5. NavHost con las tres rutas

Dentro del padding entregado por el Scaffold se declaró el NavHost con `startDestination = Screen.Home.route` y un bloque `composable(route) { ... }` para cada pantalla:

La lambda pasada a `EdificiosScreen` es la que comunica la selección hacia el Composable raíz.

```

} { paddingValues ->
    NavHost(
        navController = navController,
        startDestination = Screen.Home.route,
        modifier = Modifier.padding(paddingValues)
    ) {
        composable(Screen.Home.route) {
            HomeScreen(
                edificioSeleccionado = edificioSeleccionado
            )
        }
        composable(Screen.Edificios.route) {
            EdificiosScreen(
                onEdificioSeleccionado = { nombre ->
                    edificioSeleccionado = nombre
                    navController.navigate(Screen.Home.route) {
                        popUpTo(Screen.Home.route) {
                            include = false
                        }
                        launchSingleTop = true
                    }
                }
            )
        }
        composable(Screen.Mapa.route) {
            MapaScreen()
        }
    }
}

```

Figura 5: Paso 5. NavHost con las rutas home, edificios y mapa.

Paso 6. Implementación de HomeScreen

Se implementó `HomeScreen(edificioSeleccionado: String)` mostrando un texto de bienvenida y otro con el valor recibido:

Este Composable es completamente stateless: recibe el valor como parámetro y Compose lo redibuja automáticamente cuando el estado del Composable raíz cambia.

```

@Usage
@Composable
fun HomeScreen(
    edificioSeleccionado: String
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Bienvenido a Campus Navigator"
        )
        Text(
            text = "Último edificio consultado:"
        )
        Text(
            text = edificioSeleccionado
        )
    }
}

```

Figura 6: Paso 6. Pantalla Home con el último edificio consultado.

Paso 7. Implementación de EdificiosScreen

Se implementó `EdificiosScreen(onEdificioSeleccionado: (String) -> Unit)` con una `LazyColumn` que lista al menos 4 edificaciones (nombre + botón). Al pulsar un ítem se invoca `onEdificioSeleccionado(nombre)`:

El tipo `(String) -> Unit` define la función lambda de retorno, que desacopla la lista de edificios del estado global de la navegación.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

1 Usage
@Composable
fun EdificiosScreen(
    onEdificioSeleccionado: (String) -> Unit
) {
    val edificios = listOf(
        "Biblioteca Central",
        "Pabellón A",
        "Pabellón B",
        "Auditorio"
    )

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        items(edificios) { nombre ->
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 8.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(

```

Figura 7: Paso 7. Lista de edificios en LazyColumn con botón Ver.

Paso 8. Implementación de MapaScreen

Se implementó MapaScreen() con un Text marcador de posición, dejando el mapa real para una posterior integración:

La lambda que actualiza edificioSeleccionado ya se declaró en el Paso 5, al definir composable(Screen.Edificios.route) dentro del NavHost; por ello no se requiere ningún parámetro adicional aquí.

```

1 Usage
@Composable
fun MapaScreen() {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = "Mapa de ubicaciones (pendiente de integrar)"
        )
    }
}

```

Figura 8: Paso 8. Pantalla Mapa con marcador de posición.

Paso 9. MainActivity invocando a MainScreen()

En MainActivity, dentro de onCreate(), se reemplazó el contenido de setContent {} (el que trae la plantilla por defecto, con Scaffold y Greeting) para invocar a MainScreen():

De esta forma, el tema de la plantilla se conserva y la navegación queda centralizada en el Composable raíz MainScreen().

```

1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            CampusNavigator_ComposeTheme {
                MainScreen()
            }
        }
    }
}

```

Figura 9: Paso 9. MainActivity llamando a MainScreen().

Paso 10. Ejecución y verificación en el emulador

Se ejecutó la aplicación en el emulador y se verificó que, al pulsar cada ítem de la barra inferior, se muestra la pantalla correspondiente conservando el resaltado del ítem activo. Luego, al seleccionar un edificio en la pestaña Edificios y volver a

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

Home, se confirmó que HomeScreen muestra el valor actualizado, lo que demuestra el correcto funcionamiento de la comunicación mediante lambda entre pantallas.

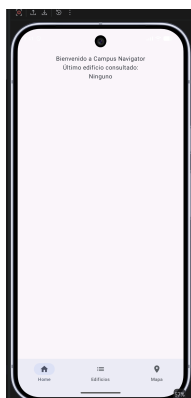


Figura 10: (a) Pantalla Home inicial («Ninguno»)

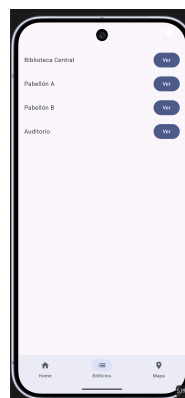


Figura 11: (b) Pantalla Edificios (Lista)

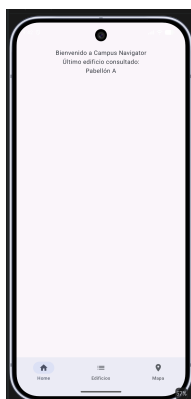


Figura 12: (c) Home tras seleccionar Pabellón A

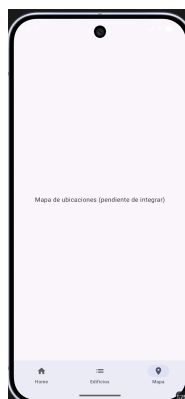


Figura 13: (d) Pantalla Mapa (Marcador)

EJERCICIOS/PROBLEMAS PROPUESTOS

Pasos:

1. Crea una clase `SeleccionViewModel : ViewModel()` con una propiedad `mutableStateOf` (o `StateFlow`) para almacenar el edificio seleccionado.

Se creó el archivo `viewModel/SeleccionViewModel.kt` con la clase `SeleccionViewModel` :

```
ViewModel(), que expone el edificio seleccionado tanto con mutableStateOf (con private
set) como con un StateFlow construido a partir de un MutableStateFlow, además de los métodos seleccionarEdificio() y
limpiarSeleccion():
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

1 package com.example.campusnavigator_compose.viewmodel
2
3 import androidx.lifecycle.ViewModel
4
5
6
7
8
9
10
11 class SeleccionViewModel : ViewModel() {
12
13     3 Usages
14     var edificioSeleccionado by mutableStateOf( value = "Ninguno")
15     private set
16
17     3 Usages
18     private val _edificioSeleccionadoFlow = MutableStateFlow( value = "Ninguno")
19
20     val edificioSeleccionadoFlow: StateFlow<String> =
21         _edificioSeleccionadoFlow.asStateFlow()
22
23     1 Usage
24     fun seleccionarEdificio(nombre: String) {
25         edificioSeleccionado = nombre
26         _edificioSeleccionadoFlow.value = nombre
27     }
28
29     fun limpiarSelección() {
30         edificioSeleccionado = "Ninguno"
31         _edificioSeleccionadoFlow.value = "Ninguno"
32     }
33 }

```

Figura 14: Clase SeleccionViewModel con el estado en mutableStateOf y en StateFlow.

En MainActivity.kt se eliminó el var edificioSeleccionado by remember { mutableStateOf("Ninguno") } que usaba la solución con lambda (y con él los imports mutableStateOf y setValue), y en libs.versions.toml más app/build.gradle.kts se agregaron lifecycle-viewmodel-ktx y lifecycle-viewmodel-compose en la versión 2.11.0, porque el proyecto todavía no tenía ninguna dependencia de ViewModel. El valor por defecto quedó además expuesto como la constante SeleccionViewModel.SIN_SELECCION, que es la que usa HomeScreen para distinguir si todavía no hay ningún edificio consultado.

2. Obtén una única instancia de SeleccionViewModel con viewModel() en el Composable raíz y pásala

a EdificiosScreen y a HomeScreen (en lugar de la lambda del ejercicio resuelto).

En el Composable raíz MainScreen se obtiene una única instancia con viewModel() y esa misma instancia se entrega a las dos pantallas dentro del NavHost, en lugar de la lambda del ejercicio resuelto. Como viewModel() conserva la instancia por ViewModelStoreOwner (el Activity), ambas pantallas reciben exactamente el mismo objeto aunque se abran y cierren varias veces:

```

1 @Composable
2 fun MainScreen() {
3
4     val navController = rememberNavController()
5
6     val seleccionViewModel: SeleccionViewModel = viewModel()
7
8     val items = listOf( Screen.Home, Screen.Edificios, Screen.Mapa, Screen.Perfil )
9
10    Scaffold(
11        bottomBar = { /* NavigationBar con un NavigationBarItem por screen */ }
12    ) { paddingValues ->
13
14        NavHost(
15            navController = navController,
16            startDestination = Screen.Home.route,
17            modifier = Modifier.padding(paddingValues)
18        ) {
19
20            composable(Screen.Home.route) {
21
22                HomeScreen(
23                    viewModel = seleccionViewModel
24                )
25            }
26        }
27    }
28 }

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```

27         composable(Screen.Edificios.route) {
28
29             EdificiosScreen(
30                 viewModel = seleccionViewModel,
31                 navController = navController
32             )
33         }
34
35         // composable(Screen.Mapa.route) { MapaScreen() }
36         // composable(Screen.Perfil.route) { PerfilScreen() }
37     }
38 }
39 }

```

El nuevo import `androidx.lifecycle.viewmodel.compose.viewModel` es el único que se requirió en este punto. El resto del ajuste se detalla en el punto 3, donde se escriben las nuevas firmas de los Composables.

3. En EdificiosScreen, al seleccionar un edificio, actualiza directamente el valor en el ViewModel

en vez de invocar una función lambda.

Recibida la instancia compartida del punto 2, EdificiosScreen cambió su firma de `onEdificioSeleccionado: (String) -> Unit` a `viewModel: SeleccionViewModel` más `navController: NavHostController`, y el botón **Ver** actualiza el ViewModel antes de navegar en lugar de invocar la lambda:

```

1  @Composable
2  fun EdificiosScreen(
3      viewModel: SeleccionViewModel,
4      navController: NavHostController
5  ) {
6      // ...
7      items(edificios) { edificio ->
8          // ...
9          FilledTonalButton(
10             onClick = {
11
12                 viewModel.seleccionarEdificio(edificio.nombre)
13
14                 navController.navigate(Screen.Home.route) {
15                     popUpTo(Screen.Home.route) {
16                         inclusive = false
17                     }
18                     launchSingleTop = true
19                 }
20             }
21         ) {
22
23             Text("Ver")
24         }
25     }
26 }

```

La lambda ya no existe en ningún lado: la pantalla conversa directamente con el ViewModel. Se llamó al método `seleccionarEdificio()` en vez de asignar la propiedad porque el `private set` impide escribirla desde la pantalla y una asignación directa dejaría el `StateFlow` desincronizado. Como único import nuevo se agregó `androidx.navigation.NavHostController`, necesario para el tipo del parámetro.

4. En HomeScreen, lee el valor del ViewModel (con `collectAsState()`) si usaste `StateFlow` y muéstralo,

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

verificando que el comportamiento visual sea el mismo que con la lambda.

En HomeScreen se dejó de leer la propiedad del ViewModel directamente y se declaró una variable local que **observa** el StateFlow mediante collectAsState(), que es la que muestra la tarjeta de «Último edificio consultado». Solo se agregó el import de collectAsState (getValue ya estaba) y el archivo SeleccionViewModel.kt no se modificó:

```

1 @Composable
2 fun HomeScreen(
3     viewModel: SeleccionViewModel
4 ) {
5
6     val edificioSeleccionado by viewModel.edificioSeleccionadoFlow.collectAsState()
7
8     val sinSeleccion = edificioSeleccionado == SeleccionViewModel.SIN_SELECCION
9
10    // ...
11
12    Card(
13        // ...
14    ) {
15
16        // ...
17
18        Text(
19            text = "Último edificio consultado",
20            style = MaterialTheme.typography.labelMedium,
21
22            color = MaterialTheme.colorScheme.onSecondaryContainer
23        )
24
25        Text(
26            text = edificioSeleccionado,
27            style = MaterialTheme.typography.titleMedium,
28
29            color = MaterialTheme.colorScheme.onSecondaryContainer
30        )
31    }
32 }
```

El valor leído además permite alternar el ícono de la tarjeta entre búsqueda y ubicación mediante sinSeleccion, es decir, la suscripción al flujo también alimenta elementos de la interfaz distintos del texto.

La verificación se hizo en el emulador con uiautomator dump (no se pudieron revisar las capturas). El texto bajo el rótulo «Último edificio consultado» fue:

```

1 Último edificio consultado: → Ninguno
2 Último edificio consultado: → Biblioteca Central
3 Último edificio consultado: → Pabellón B
```

Idéntico a la versión con lambda: mismo valor inicial, misma navegación automática a Home y actualización del valor en vivo. Se observaron dos detalles: el valor está duplicado en mutableStateOf y en el StateFlow (bastaría con el StateFlow como única fuente de verdad), y collectAsState() no es lifecycle-aware, por lo que en producción corresponde collectAsStateWithLifecycle() con la dependencia lifecycle-runtime-compose.

5. Agrega una cuarta pestaña «Perfil» (u otra que prefieras) con un Composable simple de marcador de posición, y su NavigationBarItem correspondiente.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

Se creó el objeto Screen.Perfil en la sealed class con su route y su label, se registró su NavigationBarItem en la barra inferior y se añadió composable(Screen.Perfil.route) en el NavHost para invocar a PerfilScreen, un Composable simple de marcador de posición. La navegación inferior queda con cuatro destinos y el resaltado del ítem activo se mantiene porque sigue dependiendo de la ruta actual de la pila de navegación.

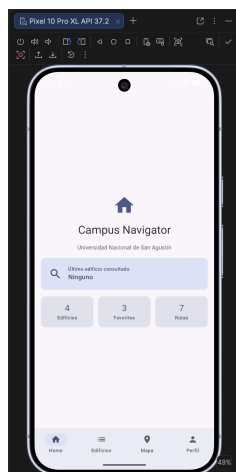


Figura 15: (a) Pestaña Home («Ninguno»)

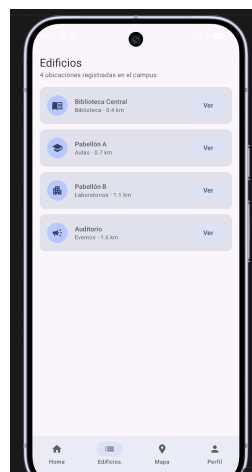


Figura 16: (b) Pestaña Edificios

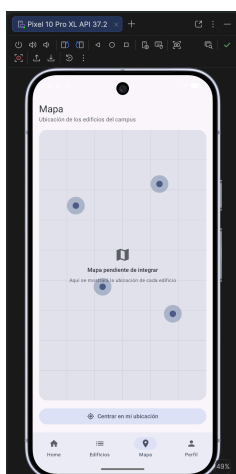


Figura 17: (c) Pestaña Mapa

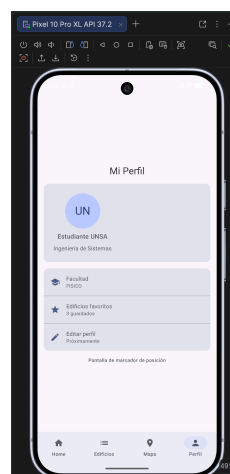


Figura 18: (d) Cuarta pestaña Perfil

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1: ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantallas Home, Edificios y Mapa navegables mediante NavigationBar, donde el edificio seleccionado en Edificios se refleja en Home)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.
Sí, se resolvió completamente. La NavigationBar inferior navega entre Home, Edificios y Mapa marcando siempre el ítem activo, y el edificio elegido en Edificios se refleja en Home, que pasa de mostrar «Ninguno» a mostrar el nombre seleccionado. Las capturas del emulador evidencian el recorrido completo de la solución.

Ejemplo: se pulsa **Ver** en el «Pabellón A» de la pestaña **Edificios** y, al volver a **Home**, el texto ya indica «Último edificio: Pabellón A».

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

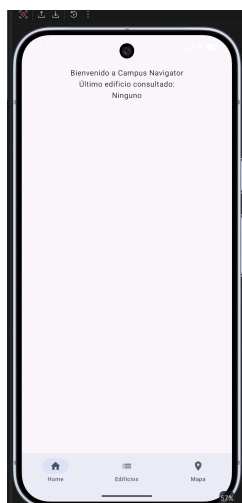


Figura 19: (a) Home inicial («Ninguno»)

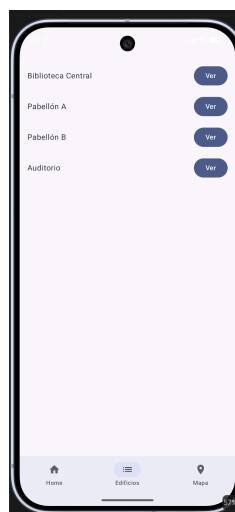


Figura 20: (b) Edificios (selección)

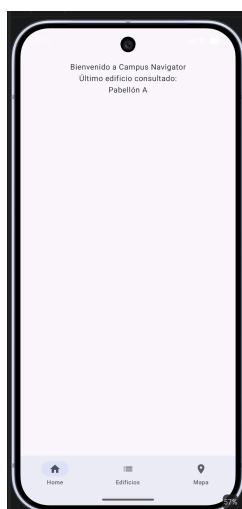


Figura 21: (c) Home ya actualizado

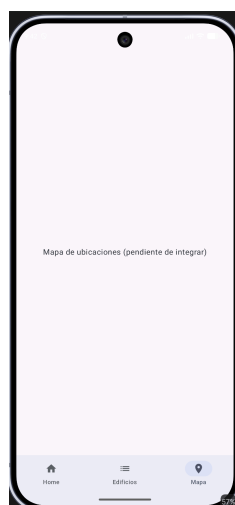


Figura 22: (d) Pantalla Mapa

Pregunta 2: En tu propia implementación, ¿qué diferencia encontraste entre comunicar Composables mediante una función lambda y hacerlo mediante un ViewModel compartido? ¿Cuál de las dos alternativas consideras una mejor práctica? Explica brevemente los motivos.

La lambda solo notifica el evento al Composable raíz, que guarda el estado en mutableStateOf; es simple, pero ese estado se pierde al recrear la Activity. El ViewModel compartido conserva el estado entre pantallas, rotaciones y cambios de configuración. Para este laboratorio basta la lambda, pero en una app real el ViewModel es mejor práctica: desacopla la UI y es más fácil de testear.

Ejemplo: el mismo edificio notificado de dos formas.

```

1 // Lambda: el estado vive en la UI y se pierde al recrear la pantalla
2 var edificio by remember { mutableStateOf("Ninguno") }
3 EdificiosScreen(onEdificioSeleccionado = { edificio = it })
4

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```

5 // ViewModel compartido: el estado sobrevive a la navegación y a la rotación
6 class EdificioViewModel : ViewModel() {
7     var edificio by mutableStateOf("Ninguno"); private set
8     fun seleccionar(nombre: String) { edificio = nombre }
9 }

```

Pregunta 3: En forma individual, explica qué ventaja adicional ofrece un ViewModel frente a otras formas de mantener el estado al navegar entre pantallas. (Cada integrante del grupo debe incluir su propia respuesta en el informe).

Frente a remember o a un objeto object global, el ViewModel mantiene el estado vivo durante toda la navegación y ante rotaciones o cambios de configuración, sin recrearse ni filtrar memoria. Además, al usar viewModelScope, las corrutinas se cancelan solas cuando la pantalla desaparece, lo que evita operaciones innecesarias y permite inyectar dependencias para las pruebas.

Ejemplo: si el usuario elige «Pabellón A» en Edificios y luego gira el teléfono, remember perdería el valor, pero el ViewModel seguiría mostrando «Pabellón A» en Home.

```

1 class EdificioViewModel : ViewModel() {
2     var edificio by mutableStateOf("Ninguno"); private set
3     fun seleccionar(nombre: String) {
4         viewModelScope.launch { /* simular llamada a API o BD */ }
5         edificio = nombre
6     }
7 }

```

III. CONCLUSIONES

- Redacta aquí la primera conclusión de la práctica.
- Redacta aquí la segunda conclusión de la práctica.
- Redacta aquí la tercera conclusión de la práctica.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.
- [2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.
- [3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.